

Sistema de Gestão de Navios Petroleiros

SMF — Ship Management Fleet

Trabalho de Avaliação em Grupo — Tópicos de Bases de Dados

Instituto Superior de Engenharia do Porto (ISEP)

Unidade Curricular: Tópicos de Bases de Dados — 2025/2026

Junho de 2026

1. Introdução

Este trabalho tem como objectivo o desenvolvimento de um sistema de gestão de operações marítimas para uma empresa de transporte de petróleo e derivados. O sistema, designado SMF (*Ship Management Fleet*), permite registar navios, planejar viagens, associar cargas e gerir a tripulação, garantindo que todas as operações respeitam as regras de negócio definidas no enunciado.

A aplicação foi desenvolvida em Java 17 com interface gráfica em JavaFX 21, ligada a uma base de dados Microsoft SQL Server alojada no Azure. A arquitectura segue o modelo de três camadas — modelo de domínio, acesso a dados (DAL) e lógica de negócio (BLL) — com separação clara de responsabilidades entre cada camada.

2. Decisões de Modelação

2.1 Estrutura da base de dados

A base de dados é composta por sete tabelas: **navios**, **cargas**, **tripulantes**, **portos**, **viagens**, e duas tabelas de associação — **viagem_carga** e **viagem_tripulante**. Estas últimas implementam as relações de muitos-para-muitos entre viagens e cargas, e entre viagens e tripulantes, respectivamente. Cada tabela de associação contém apenas os identificadores das entidades envolvidas, o que simplifica as operações de inserção e remoção.

2.2 Tipos de navios e compatibilidade com cargas

Foram definidos quatro tipos de navios — CRUDE, REFINADO, QUIMICO e QUIMICO_PRODUTO — e seis tipos de carga — PETROLEO_BRUTO, GASOLINA, DIESEL, JET_FUEL, FUELOLEO e PRODUTO_QUIMICO. A compatibilidade entre tipos de navio e tipos de carga foi implementada como um método na enumeração *TipoNavio*, centralizando esta regra num único sítio e evitando condicionais dispersas pelo código.

2.3 Ciclo de vida das viagens

O ciclo de vida de uma viagem segue uma progressão unidireccional: PLANEADA → EM_CURSO → CONCLUIDA. O estado CANCELADA pode ser atingido a partir de PLANEADA ou EM_CURSO, mas nunca a partir de CONCLUIDA. Esta restrição foi implementada na camada de lógica de negócio (*ViagemService*), por ser mais flexível e testável, e complementada por um trigger na base de dados.

2.4 Padrão Factory

Para facilitar a criação de navios e cargas com configurações correctas, foram implementadas duas classes factory — *NavioFactory* e *CargaFactory*. Cada método garante que o objecto criado tem os atributos de segurança (inflamável, corrosivo, tóxico) adequados ao tipo em questão, reduzindo o risco de erros na criação manual.

2.5 Gestão de credenciais

As credenciais de acesso à base de dados são lidas de um ficheiro `.env` armazenado em `src/main/resources` e excluído do controlo de versão através do `.gitignore`, evitando a exposição de informação sensível no repositório.

3. Componentes da Base de Dados

3.1 Vistas

Foram criadas quatro vistas para facilitar a consulta de informação agregada:

vw_viagens_ativas — lista todas as viagens no estado PLANEADA ou EM_CURSO, com o nome do navio e os portos de origem e destino. Útil para monitorizar operações em curso.

vw_navios_disponiveis — apresenta os navios com estado ATIVO que não têm viagem activa. Permite identificar rapidamente quais os navios disponíveis para nova missão.

vw_historico_tripulante — mostra todas as viagens em que cada tripulante participou, incluindo datas e portos. Suporta a funcionalidade de histórico de participação.

vw_cargas_por_viagem — lista as cargas associadas a cada viagem, com tipo, peso e porto de descarga. Facilita a verificação da carga total transportada.

3.2 Funções

Foram definidas três funções escalares reutilizáveis em queries e procedures:

fn_peso_total_viagem(viagemId) — devolve a soma do peso de todas as cargas associadas a uma viagem.

fn_capacidade_disponivel(viagemId) — devolve a diferença entre a capacidade máxima do navio e o peso total já carregado.

fn_navio_disponivel(navioId) — devolve 1 se o navio não tem viagem activa, e 0 caso contrário.

3.3 Stored Procedures

Foram criadas quatro stored procedures que encapsulam as operações principais:

sp_criar_viagem — cria uma nova viagem após validar que o navio está ATIVO e sem viagem activa. Define o estado inicial como PLANEADA.

sp_avancar_estado_viagem — avança o estado de uma viagem respeitando a máquina de estados, lançando erro se a transição não for válida.

sp_associar_carga_viagem — associa uma carga verificando compatibilidade com o tipo de navio e garantindo que a capacidade máxima não é excedida.

sp_cancelar_viagem — cancela uma viagem se estiver em PLANEADA ou EM_CURSO, recusando o pedido se já estiver CONCLUIDA ou CANCELADA.

4. Triggers e Observações

4.1 Triggers

Foram implementados três triggers para garantir integridade ao nível da base de dados, como complemento às validações feitas na aplicação:

trg_validar_compatibilidade_carga — activa antes de inserção em *viagem_carga*. Verifica se o tipo de carga é compatível com o tipo do navio. Se não for, impede a inserção.

trg_validar_capacidade_navio — activa antes de inserção em *viagem_carga*. Calcula o peso total das cargas já associadas e rejeita a operação se a capacidade máxima do navio for excedida.

trg_bloquear_edicao_viagem_activa — activa antes de actualização em *viagens*. Impede alterações aos campos principais quando a viagem está em EM_CURSO, CONCLUIDA ou CANCELADA.

4.2 Observações e dificuldades encontradas

O principal desafio técnico foi a integração entre JavaFX e Java 25, que introduziu restrições ao acesso de métodos nativos. O problema foi resolvido com a criação de uma classe *Launcher* separada que não estende *Application*, contornando a verificação prévia de módulos JavaFX efectuada pela JVM.

Um segundo problema detectado foi o bloqueio da interface gráfica durante operações à base de dados. O serviço Azure SQL gratuito entra em repouso após períodos de inactividade, e a tentativa de ligação bloqueava o JavaFX Application Thread, tornando a aplicação irresponsiva. A solução passou por mover todas as operações à base de dados para threads em segundo plano, actualizando a interface apenas após a conclusão, através de *Platform.runLater()*.

A suite de testes JUnit inclui 155 testes automáticos que cobrem as camadas de modelo, factory e lógica de negócio, verificando todos os fluxos definidos no enunciado, incluindo a máquina de estados das viagens, as regras de compatibilidade e as restrições de capacidade. Todos os testes passam sem falhas.

4.3 Conclusão

O sistema SMF cumpre os requisitos funcionais e técnicos definidos no enunciado. A separação em camadas facilita a manutenção e evolução do código. As regras de negócio estão implementadas tanto na aplicação Java como na base de dados, através de stored procedures e triggers, garantindo a integridade dos dados independentemente da origem das operações.